

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

NAME: A. Suraj varma

Reg.No : 192525339

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)

Assessment Tool Weightage: 30%

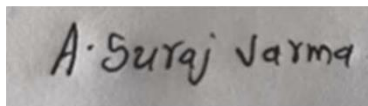
QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.	BL3 – Apply	5
2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.	BL3 – Apply	5
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.	BL3 – Apply	5
4	Debug the given relational algebra expression that produces incorrect output for a natural join operation.	BL3 – Apply	5
5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.	BL3 – Apply	5
6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.	BL6 – Create	5
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.	BL3 – Apply	5

8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.	BL3 – Apply	5
9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes.	BL6 – Create	5
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.	BL3 – Apply	5

Code of Conduct:

I A. Suraj Varma (192525339) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated

Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation
--------------	-----	---	--	---	---

Classroom Allotment System

Q1. Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.

Answer:

The error occurs because the tables are joined without a proper join condition. This creates a Cartesian product and produces duplicate or unrelated results.

Incorrect query:

```
SELECT f.FacultyName, c.CourseName, c.HallNo
FROM faculty f, course c;
```

The above query does not specify how FACULTY and COURSE are related.

Correct query:

```
SELECT
    f.FacultyID,
    f.FacultyName,
    c.CourseName,
    c.HallNo
FROM faculty f
INNER JOIN course c
ON f.FacultyID = c.FacNo;
```

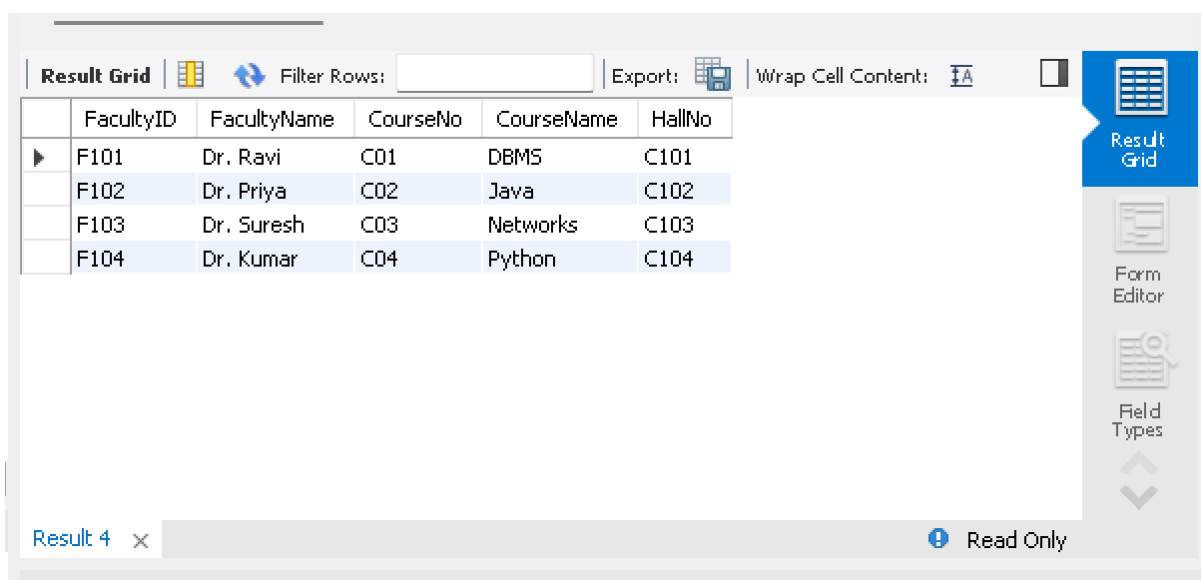
Explanation:

The FACULTY table contains FacultyID and the COURSE table contains FacNo. These columns are used to connect a faculty member with the course assigned to that faculty member.

The INNER JOIN returns only the matching records and avoids the unnecessary duplicate combinations.

Result:

The query correctly displays the faculty name, course name and classroom/hall number.



The screenshot shows a database application window with a 'Result Grid' tab selected. The grid displays the results of an SQL query. The columns are FacultyID, FacultyName, CourseNo, CourseName, and HallNo. There are four rows of data. The interface includes a 'Filter Rows' field, an 'Export' button, and a 'Wrap Cell Content' checkbox. On the right side, there are buttons for 'Result Grid', 'Form Editor', and 'Field Types'. At the bottom, there is a 'Read Only' status indicator.

	FacultyID	FacultyName	CourseNo	CourseName	HallNo
▶	F101	Dr. Ravi	C01	DBMS	C101
	F102	Dr. Priya	C02	Java	C102
	F103	Dr. Suresh	C03	Networks	C103
	F104	Dr. Kumar	C04	Python	C104

Q2. The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.

Answer:

The problem is that the sub-query calculates the average number of students for all faculty members. It does not compare the faculty member with the average of their own branch.

Incorrect query:

```
SELECT *  
FROM faculty f1  
WHERE f1.NoOfStudents >  
(  
    SELECT AVG(f2.NoOfStudents)  
    FROM faculty f2  
);
```

Correct query:

```
SELECT  
    f1.FacultyID,  
    f1.FacultyName,  
    f1.Branch,  
    f1.NoOfStudents  
FROM faculty f1  
WHERE f1.NoOfStudents >  
(  
    SELECT AVG(f2.NoOfStudents)  
    FROM faculty f2  
    WHERE f1.Branch = f2.Branch  
);
```

Explanation:

The condition

WHERE f1.Branch = f2.Branch

creates the correlation between the outer query and the inner query.

Now each faculty member is compared with the average number of students in the same branch.

Result:

The query displays faculty members whose number of students is greater than the average number of students for their branch.

	FacultyID	FacultyName	Branch	NoOfStudents
▶	F101	Dr. Ravi	CSE	50
•	NULL	NULL	NULL	NULL

Q3. A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.

Answer:

The error occurs because the view is using a column that does not exist in the FACULTY table.

Incorrect view:

```
CREATE VIEW faculty_classroom AS
SELECT FacultyName, ClassroomName
FROM faculty;
```

The column ClassroomName does not exist in the FACULTY table. The correct column is Classroom.

Correct and optimized view:

```
DROP VIEW IF EXISTS faculty_classroom;
```

```
CREATE VIEW faculty_classroom AS
SELECT
```

```
    FacultyID,
    FacultyName,
    Subject,
    Classroom,
    NoOfStudents
```

```
FROM faculty;
```

To display the view:

```
SELECT *
FROM faculty_classroom;
```

Explanation:

The corrected view uses only valid columns from the FACULTY table. It displays the important information required for classroom allotment.

The view is optimized by selecting only the required columns instead of using unnecessary data.

Result:

The view correctly displays faculty ID, faculty name, subject, classroom and number of students.

	FacultyID	FacultyName	Subject	Classroom	NoOfStudents
►	F101	Dr. Ravi	DBMS	C101	50
	F102	Dr. Priya	Java	C102	45
	F103	Dr. Suresh	Networks	C103	40
	F104	Dr. Kumar	Python	C104	35

Q4. Debug the given relational algebra expression that produces incorrect output for a natural join operation.

Answer:

A natural join automatically joins relations using attributes having the same name.

In the Classroom Allotment database:

FACULTY contains:

FacultyID

COURSE contains:

FacNo

Although both represent the faculty ID, their attribute names are different.

Incorrect relational algebra expression:

$FACULTY \bowtie COURSE$

The natural join cannot correctly match FacultyID with FacNo.

Correct relational algebra expression:

$FACULTY \bowtie \rho(FacultyID \leftarrow FacNo)(COURSE)$

The rename operation changes FacNo to FacultyID before performing the natural join.

The relationship becomes:

$FACULTY.FacultyID = COURSE.FacNo$

Equivalent SQL query:

SELECT

f.FacultyID,
f.FacultyName,
c.CourseName,
c.HallNo

FROM faculty f

INNER JOIN course c

ON f.FacultyID = c.FacNo;

Result:

The faculty and course information is correctly combined using the faculty ID.

Correct Relational Algebra: $FACULTY \bowtie \rho(FacultyID \leftarrow FacNo)(COURSE)$

	FacultyID	FacultyName	CourseNo	CourseName	HallNo
►	F101	Dr. Ravi	C01	DBMS	C101
	F102	Dr. Priya	C02	Java	C102
	F103	Dr. Suresh	C03	Networks	C103
	F104	Dr. Kumar	C04	Python	C104

Q5. The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.

Answer:

The referential integrity problem occurs when a course contains a FacNo that does not exist in the FACULTY table.

For example:

```
INSERT INTO course
VALUES
```

```
('C05', 'Artificial Intelligence', 'Theory', 5, 'C105', 'F999');
```

Here, F999 does not exist in the FACULTY table.

Therefore, the COURSE table contains an invalid faculty reference.

First, the FACULTY table should have a primary key:

```
ALTER TABLE faculty
```

```
ADD PRIMARY KEY (FacultyID);
```

Then create a foreign key in the COURSE table:

```
ALTER TABLE course
```

```
ADD CONSTRAINT fk_course_faculty
```

```
FOREIGN KEY (FacNo)
```

```
REFERENCES faculty(FacultyID);
```

Now a valid record can be inserted:

```
INSERT INTO course
VALUES
```

```
('C05', 'Artificial Intelligence', 'Theory', 5, 'C105', 'F101');
```

An invalid record such as the following will not be accepted:

```
INSERT INTO course
VALUES
```

```
('C06', 'Cloud Computing', 'Theory', 5, 'C106', 'F999');
```

Explanation:

The foreign key ensures that every FacNo in COURSE must match an existing FacultyID in FACULTY.

This maintains referential integrity and prevents invalid faculty references.

	CourseNo	CourseName	CourseType	SemNo	HallNo	FacNo
▶	C01	DBMS	Theory	3	C101	F101
	C02	Java	Theory	3	C102	F102
	C03	Networks	Theory	4	C103	F103
	C04	Python	Lab	4	C104	F104
	C05	Artificial Intelligence	Theory	5	C105	F101
*	NULL	NULL	NULL	NULL	NULL	NULL

Q6. Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.

Answer:

The nested sub-query can be replaced with an INNER JOIN. This makes the relationship between COURSE and FACULTY clearer and can improve query performance.

Inefficient query:

```
SELECT
    CourseName,
    HallNo
FROM course
WHERE FacNo IN
(
    SELECT FacultyID
    FROM faculty
    WHERE Branch = 'CSE'
);
```

Optimized query using JOIN:

```
SELECT
    c.CourseNo,
    c.CourseName,
    c.HallNo,
    f.FacultyName
FROM course c
INNER JOIN faculty f
ON c.FacNo = f.FacultyID
WHERE f.Branch = 'CSE';
```

Explanation:

The JOIN directly connects the COURSE and FACULTY tables using:

c.FacNo = f.FacultyID

The branch condition is then applied to the faculty table.

This avoids unnecessary nested searching and provides a simpler and more efficient query.

Result:

The query displays courses and halls handled by CSE faculty members.

	CourseNo	CourseName	HallNo	FacultyName
▶	C01	DBMS	C101	Dr. Ravi
	C05	Artificial Intelligence	C105	Dr. Ravi
	C03	Networks	C103	Dr. Suresh

Q7. The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.

Answer:

The error occurs because FacultyName is selected but is not included in the GROUP BY clause and is not used with an aggregate function.

Incorrect query:

```
SELECT
    Branch,
    FacultyName,
    COUNT(*) AS Total_Faculty
FROM faculty
GROUP BY Branch;
```

The query groups records only by Branch but also tries to display FacultyName.

Correct query:

```
SELECT
    Branch,
    COUNT(*) AS Total_Faculty
FROM faculty
GROUP BY Branch;
```

Explanation:

The GROUP BY Branch creates one group for each branch.

The COUNT(*) function counts the number of faculty members in each branch.

Another useful classroom allotment query is:

```
SELECT
    Classroom,
    COUNT(*) AS Faculty_Count
FROM faculty
GROUP BY Classroom;
```

Result:

The query correctly displays the number of faculty members in each branch or classroom.

	Branch	Total_Faculty
▶	CSE	2
	AIML	1
	ECE	1

Q8. Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.

Answer:

The error occurs because the UPDATE statement does not contain a WHERE condition.

Incorrect query:

```
UPDATE faculty
```

```
SET Classroom = 'C105';
```

This statement changes the classroom of every faculty member in the table.

Correct query:

```
UPDATE faculty
```

```
SET Classroom = 'C105'
```

```
WHERE FacultyID = 'F101';
```

To verify the updated record:

```
SELECT *
```

```
FROM faculty
```

```
WHERE FacultyID = 'F101';
```

Explanation:

The WHERE condition identifies the particular faculty member whose classroom should be changed.

Without the WHERE condition, all records are modified.

Best practice:

Before updating, check the required record:

```
SELECT *
```

```
FROM faculty
```

```
WHERE FacultyID = 'F101';
```

Then execute the UPDATE statement.

Result:

Only faculty member F101 is assigned to classroom C105.

	FacultyID	FacultyName	Classroom
▶	F101	Dr. Ravi	C101
	F102	Dr. Priya	C102
	F103	Dr. Suresh	C103
	F104	Dr. Kumar	C104
✱	NULL	NULL	NULL

Q9. Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes.

Answer:

A multi-table query can become inefficient when several nested sub-queries are used. JOIN can directly connect related tables.

Inefficient query:

```
SELECT
    CourseName,
    HallNo
FROM course
WHERE FacNo IN
(
    SELECT FacultyID
    FROM faculty
    WHERE Branch = 'CSE'
);
```

Optimized query:

```
SELECT
    f.FacultyID,
    f.FacultyName,
    f.Branch,
    c.CourseNo,
    c.CourseName,
    c.HallNo
FROM faculty f
INNER JOIN course c
ON f.FacultyID = c.FacNo
WHERE f.Branch = 'CSE';
```

Create indexes for frequently searched and joined columns:

```
CREATE INDEX idx_faculty_branch
ON faculty(Branch);
```



```
CREATE INDEX idx_course_facno
ON course(FacNo);
```

	FacultyID	FacultyName	Classroom
▶	F101	Dr. Ravi	C101
	F102	Dr. Priya	C102
	F103	Dr. Suresh	C103
	F104	Dr. Kumar	C104
⬇	NULL	NULL	NULL

```
CREATE INDEX idx_course_facno
ON course(FacNo);
```

Explanation:

The JOIN directly connects the FACULTY and COURSE tables.

The index on Branch helps in searching faculty members by branch.

The index on FacNo helps in matching courses with faculty members.

Therefore, the query becomes clearer and more efficient for larger amounts of data.

Result:

The query displays the CSE faculty members along with their assigned courses and halls.

Q10. Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.

Answer:

The problem is the incorrect use of the universal quantifier $\forall$.

The universal quantifier means **for all**. If it is used when only one matching faculty member is required, the condition becomes too restrictive and may return an empty result.

Incorrect relational calculus expression:

```
{ c |  
  COURSE(c) AND  
   $\forall f$  (FACULTY(f)  $\rightarrow$  c.FacNo = f.FacultyID)  
}
```

This expression requires the course's faculty ID to match every faculty member.

Correct expression using the existential quantifier:

```
{ c |  
  COURSE(c) AND  
   $\exists f$  (FACULTY(f) AND  
    c.FacNo = f.FacultyID AND  
    f.Branch = 'CSE')  
}
```

Here, $\exists$ means **there exists**.

The expression finds courses for which there exists a faculty member whose FacultyID matches the course's FacNo and whose branch is CSE.

Equivalent SQL query:

```
SELECT  
  c.CourseNo,  
  c.CourseName,  
  c.HallNo  
FROM course c  
INNER JOIN faculty f  
ON c.FacNo = f.FacultyID  
WHERE f.Branch = 'CSE';
```

Result:

The corrected expression returns the courses handled by CSE faculty members instead of returning an empty result due to the incorrect universal condition.

	CourseNo	CourseName	HallNo	FacultyName
►	C01	DBMS	C101	Dr. Ravi
	C05	Artificial Intelligence	C105	Dr. Ravi
	C03	Networks	C103	Dr. Suresh